# Category 1: Fragmentation and Data Distribution

### 1. PHF Analyzer: The "Global Logistics" Dataset

- **Dataset:** A Shipments table with attributes: ShipID, Origin_Country, Weight, Priority, and Status.
- **The Task:** Implement PHF using a predicate set $Pr = \{Origin\_Country = 'USA', Origin\_Country = 'EU', Weight > 500\}$.
- **Analysis:** Calculate all 8 possible minterm predicates. Identify which are "contradictory" (e.g., if a shipment cannot be from both USA and EU) and eliminate them to find the set of complete and minimal fragments.
- **Deliverable:** A mapping showing which ShipIDs from a 1,000-row sample fall into which fragment.

### 2. DHF Optimizer: "Retail Supply Chain"

- **Dataset:** Two tables: Suppliers (Owner) and Parts (Member), linked by SuppID.
- **The Task:** First, fragment Suppliers horizontally by Region (North, South, East, West). Then, perform Derived Horizontal Fragmentation on Parts.
- **Analysis:** Using a 5,000-row Parts dataset, analyze the "Join Selectivity." Does fragmenting Parts based on Suppliers reduce the amount of data transferred during a "List all parts by Region" query?
- **Metric:** Compare the cost of a distributed join vs. a localized join.

### 3. BEA via "Hospital Patient Records"

- **Dataset:** A wide Patient table (15 columns): ID, Name, Address, BloodType, LastVisit, Diagnosis, InsuranceID, BillingBalance, etc.
- **The Task:** Define two applications: (App A) Medical Staff looking at health data, and (App B) Billing Dept looking at financial data.
- **Analysis:** Construct the **Attribute Usage Matrix**. Use the Bond Energy Algorithm to cluster attributes.
- **Deliverable:** Identify the optimal "split point" to create two vertical fragments that minimize cross-fragment access.

### 4. Hybrid Fragmentation: "Real Estate Listings"

- **Dataset:** Properties table with attributes: PropertyID, City, Price, SquareFootage, OwnerContact, TaxHistory.
- **The Task:** Apply vertical fragmentation to separate public data (Price, City) from private data (OwnerContact, TaxHistory). Then, apply horizontal fragmentation to the public fragment by Price brackets $(< \$250k, \$250k - \$750k, > \$750k)$.
- **Analysis:** Draw the fragmentation tree. Prove that the **Reconstruction** property holds by writing the SQL JOIN + UNION query.

### 5. Predicate Tester: "University Enrollment"

- **Dataset:** Students table with GPA, Major, Year, and Credits.
- **The Task:** Students are given a messy set of predicates: $\{GPA > 3.5, Major = ' CS', GPA \leq 3.5, Year = 'Senior'\}$.
- **Analysis:** Use the **Minimality** rule from the text. A predicate is only minimal if at least one application accesses the resulting fragments differently.
- **Deliverable:** A logic report showing which predicates are redundant and a "cleaned" set of predicates that satisfy Özsu's completeness criteria.

### 6. Best-Fit Allocation: "Global News Agency"

- **Dataset:** 10 fragments of NewsArticles (organized by Category) and 4 Sites (New York, London, Tokyo, Mumbai).
- **The Task:** Provide a "Query Volume Matrix" (e.g., Tokyo reads 'Tech' fragments 500x/day; NY reads 'Finance' 1000x/day).
- **Analysis:** Use a greedy heuristic to allocate fragments to sites to minimize the total $Communication\_Cost = \sum(Frequency \times DataSize)$.
- **Metric:** A table showing the final allocation and the total network "cost" units.

### 7. Reconstruction Validator: "Vehicle Telematics"

- **Dataset:** A 10,000-row CSV of VehicleLogs (Timestamp, VIN, Speed, FuelLevel, Location, EngineTemp).
- **The Task:** Programmatically split the CSV into 4 horizontal fragments (by VIN range) and 2 vertical fragments (Operational vs. Diagnostic data).
- **Analysis:** Delete 5 random rows from one fragment. Run a "Reconstruction Script."
- **Deliverable:** The script must flag exactly which records are missing or corrupted, proving the student understands the **Lossless Join/Union** requirements.

### 8. Affinity vs. Performance: "E-Commerce CRM"

- **Dataset:** A Customer table with 20 attributes.
- **The Task:** Create two different vertical fragmentation designs: one based on "Intuition" and one based on the **Attribute Affinity Matrix**.
- **Analysis:** Simulate 100 queries. Calculate the **Discriminatory Power** of your fragmentation.
- **Metric:** Plot a chart showing "Unnecessary Attributes Accessed" for Design A vs. Design B.

### 9. Constraint-Based Allocation: "Streaming Metadata"

- **Dataset:** MovieMetadata fragments and 3 Cloud Regions (AWS-East, Azure-West, GCP-Europe).
- **The Task:** Assign fragments but with a twist: Site 1 has a 50GB storage limit, Site 2 has a high-cost-per-GB, and Site 3 has low bandwidth.
- **Analysis:** Formulate this as an optimization problem. The student must justify their allocation based on these physical constraints.
- **Deliverable:** A "Cost-Benefit" report for the chosen allocation.

### 10. Drift Detector: "Online Banking Ledger"

- **Dataset:** A Transactions table initially fragmented by BranchID.
- **The Task:** Provide a "Day 1" workload and a "Day 30" workload where customers from Branch A start using Branch B's ATMs significantly more often.
- **Analysis:** Calculate the **Locality of Reference (LR)** score for both days.
- **Metric:** The student must define the "Threshold" at which the system should trigger a re-fragmentation (e.g., when $LR < 0.7$.

Grading : Category 1

| Criteria | Excellent (90-100%) | Satisfactory (70-89%) | Developing (<70%) |
|---|---|---|---|
| **Fragmentation Logic** | Correct implementation of Primary/Derived horizontal or Vertical fragmentation. | Logic works but has minor data overlaps or gaps. | Data is incorrectly split; loss of integrity. |
| **Reconstruction** | Flawless "Inverse" operation (Union/Join) to recreate the global relation. | Reconstruction works but is highly inefficient. | Cannot reconstruct the original table from fragments. |
| **Allocation Strategy** | Justified placement of fragments based on access patterns (e.g., FDD). | Fragments are placed randomly without theoretical backing. | No clear allocation logic or single-site storage. |
| **Schema Integrity** | Maintains OIDs/Primary Keys correctly across all sites. | Missing some relational constraints in fragments. | Broken relationships between distributed sites. |

# Category 2: Distributed Query Processing (DQP)

### 11. Distributed Semi-Join Implementation: "Employee-Project"

- **Dataset:** Employees (Site 1, 10,000 rows) and Assignments (Site 2, 50,000 rows).
- **The Task:** Implement a semi-join ($R \ltimes S$) to find employees working on at least one project.
- **Analysis:** Calculate the "Size Reduction Factor." The student must send only the unique EmpIDs from Site 2 to Site 1, perform the join, and send the result back.
- **Metric:** Compare total bytes transferred using a standard Join vs. a Semi-Join.

### 12. Query Localization Engine: "Regional Sales"

- **Dataset:** A global Sales table fragmented horizontally across 3 sites: Sales_North, Sales_South, and Sales_West.
- **The Task:** Write a parser that takes a global query (e.g., SELECT * FROM Sales WHERE Amount > 1000) and "localizes" it.
- **Analysis:** Implement **Selection Pushdown**. If the query specifies Region = 'North', the engine should only query Sales_North and ignore the other sites.
- **Deliverable:** A log showing which site-specific queries were generated for 5 different input SQL strings.

### 13. Distributed Nested Loop Join Simulator: "Customer-Orders"

- **Dataset:** Customers (1,000 rows) and Orders (100,000 rows) on different sites.
- **The Task:** Simulate a Page-Oriented Nested Loop Join across a high-latency link (simulate 50ms delay per packet).
- **Analysis:** Analyze the impact of "Block Size" on total execution time.
- **Metric:** Plot a graph of Execution Time vs. Network Latency (from 1ms to 200ms).

### 14. Distributed Hash Join Prototype: "Inventory-Warehouse"

- **Dataset:** Parts (Site 1) and Inventory (Site 2).
- **The Task:** Implement the "Partitioned Hash Join" logic. Both sites hash their records into $N$ buckets and ship corresponding buckets to a central coordinator (or each other).
- **Analysis:** Evaluate the "Skew" problem. What happens if one hash bucket is 10x larger than others?
- **Deliverable:** A report on how "Hash Skew" affects parallel processing time.

### 15. Bloom Filter Join Optimizer: "Subscribers & Logs"

- **Dataset:** Subscribers (Site A, 1 million rows) and WebLogs (Site B, 10 million rows).
- **The Task:** Use a Bloom Filter to minimize data transfer. Site A sends a compact bit-vector representing its UserIDs to Site B. Site B filters its WebLogs before sending matches back.
- **Analysis:** Calculate the **False Positive Rate** of the Bloom Filter and its impact on wasted bandwidth.
- **Metric:** Bytes saved vs. the size of the bit-vector ($m$ bits).

### 16. Algebraic Query Tree Visualizer: "Academic Records"

- **Dataset:** Tables for Students, Courses, Enrollments, and Professors.
- **The Task:** Build a tool that converts a SQL query into an **Initial Algebraic Query Tree** and then into an **Optimized Tree**.

- **Analysis:** Show the transformation using rules like "Pushing down Selections" and "Commutativity of Joins."
- **Deliverable:** A UI or PDF export showing the "Before" and "After" trees for a complex 4-table join.

### 17. Cost-Based Optimizer (CBO) Prototype: "Social Media"

- **Dataset:** Users and Posts. Provide metadata: Card(Users)=10k, Card(Posts)=1M, Length(UserRecord)=100 bytes.
- **The Task:** Implement a simple cost model: $Total\_Cost = (C_{cpu} \times \#inst) + (C_{io} \times \#ios) + (C_{msg} \times \#msgs) + (C_{tr} \times \#bytes)$.
- **Analysis:** Given two execution plans (e.g., Ship-Whole-Table vs. Semi-Join), the script must programmatically pick the cheaper one.
- **Metric:** A breakdown of the estimated cost vs. actual simulated execution time.

### 18. Parallel Aggregation Engine: "IoT Sensor Network"

- **Dataset:** 1 million Temperature readings fragmented across 4 "Edge" sites.
- **The Task:** Calculate AVG(Temp) and MAX(Temp) across all sites.
- **Analysis:** Implement the **Map-Reduce** style logic: local aggregation (Site Max/Sum/Count) followed by global merging.
- **Deliverable:** Prove that the results are identical to a centralized calculation while transferring $99\%$ less data.

### 19. Sort-Merge Join for Distributed Sites: "Book-Authors"

- **Dataset:** Books (Site 1) and Authors (Site 2), both unsorted.
- **The Task:** Implement a join where both sites sort their local data concurrently, then perform a "coordinated merge."
- **Analysis:** Compare the performance when Site 1 sends sorted data to Site 2 vs. both sending to a Site 3.
- **Metric:** Measure the "Speedup" achieved by sorting in parallel compared to sorting a single combined file.

### 20. Multi-way Join Optimizer: "Supply Chain Management"

- **Dataset:** 4 tables: Suppliers, Parts, Projects, and Shipments located at 4 different sites.
- **The Task:** Implement a **Greedy Join Ordering** algorithm (as described in the textbook).
- **Analysis:** The algorithm should pick the sequence of joins (e.g., $(S \bowtie P) \bowtie J$) that minimizes intermediate result sizes.
- **Deliverable:** A step-by-step trace showing the estimated size of the intermediate relation after each join step.

---

### Grading Notes for Category 2

For these projects, the **Analysis** should focus heavily on **Response Time vs. Total Cost**. In distributed systems, total cost (bandwidth) is often sacrificed to improve response time (parallelism). Students should be able to explain this trade-off in their final report.

Grading : Category 2

| Criteria | Excellent (90-100%) | Satisfactory (70-89%) | Developing (<70%) |
|---|---|---|---|
| **Query Decomposition** | Correctly breaks global queries into local sub-queries (Relational Algebra). | Decomposition works but creates redundant sub-queries. | Fails to translate global syntax to local sites. |
| **Join Optimization** | Effectively uses Semi-joins or Bloom Filters to reduce data transfer. | Uses basic joins; high network overhead for large datasets. | Inefficient "Ship-Whole-Table" approach only. |
| **Localization** | Correctly identifies and prunes irrelevant fragments (Selection Predicates). | Sends queries to all sites regardless of data location. | No localization; queries fail on fragmented data. |
| **Cost Modeling** | Provides clear analysis of $Total\_Cost = I/O + CPU + Comm$. | Mentions costs but lacks quantitative data or formulas. | No consideration of network or processing costs. |

# Category 3: Distributed Concurrency Control

### 21. Centralized 2-Phase Locking (C2PL): "Digital Banking"

- **Dataset:** Accounts (ID, Balance, BranchID). Provide a CSV with 1,000 accounts.
- **The Task:** Implement a single "Global Lock Manager" (GLM) process. All transactions, regardless of which branch they originate from, must request Read/Write locks from the GLM.
- **Analysis:** Measure the **bottleneck effect**. How does the GLM response time change as you scale from 10 to 100 concurrent transactions?
- **Deliverable:** A log showing the lock request queue and the total "Wait Time" per transaction.

### 22. Distributed 2PL with Global Deadlock Detection: "Supply Chain"

- **Dataset:** Warehouse_Stock (Site A) and Shipping_Fleet (Site B).
- **The Task:** Implement local lock managers at both sites. Create a scenario where Transaction 1 locks a Warehouse item and wants a Truck, while Transaction 2 locks a Truck and wants a Warehouse item.
- **Analysis:** Implement a **Global "Wait-For" Graph (WFG)**. The student must demonstrate the system detecting the cycle across sites and aborting one transaction.
- **Metric:** Time taken to detect the deadlock after the cycle is actually formed.

### 23. Deadlock Prevention: Wound-Wait vs. Wait-Die: "Airline Booking"

- **Dataset:** Flight_Seats (FlightNo, SeatID, Status).
- **The Task:** Implement both the **Wound-Wait** and **Wait-Die** schemes based on transaction timestamps.
- **Analysis:** Run a "High Contention" simulation where 50 users try to book the last 5 seats on a flight.
- **Metric:** Compare the number of **Transaction Restarts** between the two algorithms. Which one is "fairer" to older transactions?

### 24. Basic Timestamp Ordering (BTO): "Stock Trading Platform"

- **Dataset:** Stock_Ticker (Symbol, CurrentPrice, LastUpdated).
- **The Task:** Implement the BTO rule: if $TS(T) < W - TS(x)$, the read is rejected. Each data item must track $R - TS$ and $W - TS$.
- **Analysis:** Simulate a high-velocity stream of price updates.
- **Deliverable:** A report showing the "Abort Rate" as the frequency of updates increases.

### 25. Distributed Optimistic Concurrency Control (OCC): "Social Media Likes"

- **Dataset:** Post_Stats (PostID, LikeCount, ViewCount).
- **The Task:** Transactions have three phases: Read (local workspace), Validation (global check), and Write (commit).
- **Analysis:** Test the system in a "Read-Heavy" vs. "Write-Heavy" environment.
- **Metric:** Show that OCC outperforms Locking when conflicts are rare, but collapses when conflicts are frequent.

### 26. Multiversion Concurrency Control (MVCC): "Wiki Edit History"

- **Dataset:** Page_Content (PageID, VersionID, Content, Timestamp).

- **The Task:** Implement a system where "Readers never block Writers." When a Write occurs, a new version is created instead of overwriting the old one.
- **Analysis:** Analyze the **Storage Overhead**. If you keep the last 5 versions of every page, how does that affect query performance for "Historical" reads?
- **Deliverable:** A demonstration of a long-running "Audit" transaction reading a consistent version while 10 other transactions update the same page.

### 27. Conservative Timestamp Ordering (CTO): "Automated Manufacturing"

- **Dataset:** Assembly_Line_Steps (StepID, MachineID, Status).
- **The Task:** Implement a scheduler that *waits* to execute a transaction until it is certain that no transaction with a smaller timestamp will arrive.
- **Analysis:** This eliminates restarts but introduces **Delay**.
- **Metric:** Measure the "Average Transaction Latency" and compare it to Basic Timestamp Ordering (Project 24).

### 28. Distributed Snapshot Isolation (SI): "Retail Inventory Sync"

- **Dataset:** Product_Inventory stored across two sites: Store_Front and Back_Office.
- **The Task:** Implement Snapshot Isolation where a transaction reads from a "frozen" version of the DB.
- **Analysis:** Demonstrate the **Write Skew** anomaly (where two transactions update different items that are related by a constraint) and implement a "First-Committer-Wins" rule to prevent it.
- **Deliverable:** A trace showing how the system handles two concurrent updates to the same logical "Stock Level."

### 29. Hierarchical Locking Simulator: "Distributed File System"

- **Dataset:** A tree-structured dataset of Directories, Sub-directories, and Files.
- **The Task:** Implement **Intention Locks** (IS, IX, SIX). To lock a file, a transaction must first place intention locks on the parent directories.
- **Analysis:** Evaluate "Lock Granularity." Is it faster to lock a whole folder or 100 individual files?
- **Metric:** Total time to acquire locks for a "Bulk Update" vs. "Single File Update."

### 30. Quorum-Based Distributed Locking: "Global Profile Service"

- **Dataset:** User_Profiles replicated across 3 sites.
- **The Task:** To update a profile, a transaction must acquire a lock on a **Majority** of the sites (2 out of 3).
- **Analysis:** Simulate a "Slow Node" (add a 500ms delay to Site 3).
- **Metric:** Does the system maintain performance as long as the majority is fast? Show the response time when 1 node is down vs. when all are healthy.

Grading : Category 3

| Criteria | Excellent (90-100%) | Satisfactory (70-89%) | Developing (<70%) |
|---|---|---|---|
| **Logic Correctness** | Zero "Lost Updates" or "Dirty Reads." | Occasional logic errors in edge cases. | Failed to maintain ACID. |
| **Simulation Quality** | Successfully simulates multiple sites/threads. | Basic simulation with limited concurrency. | Single-threaded; no real concurrency. |
| **Deadlock/Conflict Handling** | Clear mechanism for detection or prevention. | Detects conflicts but handles them poorly. | System hangs on deadlocks. |
| **Analysis** | Clear graphs showing Throughput vs. Latency. | Basic numbers provided. | No quantitative data. |

---

# Category 4: Reliability and Commit Protocols

### 31. Basic 2-Phase Commit (2PC): "Cross-Bank Transfer"

- **Dataset:** Two separate SQL tables representing Bank_A_Accounts and Bank_B_Accounts.
- **The Task:** Implement a Coordinator that manages a transfer between the two banks.
- **Analysis:** Simulate a "Coordinator Crash" after sending the *Prepare* message but before the *Commit* message.
- **Deliverable:** A log file showing the state transitions (INITIAL → PREPARE → READY → COMMIT) and how the system recovers the transaction after a simulated reboot.

### 32. 2PC with Presumed Abort (PA): "E-Commerce Checkout"

- **Dataset:** Order_Header, Inventory_Stock, and Payment_Status at three different sites.
- **The Task:** Implement the **Presumed Abort** optimization where the coordinator does not need to log an "Abort" decision.
- **Analysis:** Measure the reduction in "Log Force-Writes" compared to Basic 2PC.
- **Metric:** Count the total number of disk I/O operations for 100 successful transactions vs. 100 aborted transactions.

### 33. 2PC with Presumed Commit (PC): "Global Inventory Update"

- **Dataset:** A 5,000-row Warehouse_Inventory CSV partitioned across 4 sites.
- **The Task:** Implement the **Presumed Commit** optimization, which is efficient for transactions that rarely fail.
- **Analysis:** Analyze the overhead of the "ACD" (ACK of Commit) requirement.
- **Deliverable:** A comparison chart showing network message counts between PA (Project 32) and PC (Project 33).

### 34. 3-Phase Commit (3PC) Simulator: "Multi-Resource Reservation"

- **Dataset:** Hotel_Rooms, Flight_Seats, and Car_Rentals datasets.
- **The Task:** Implement the "Pre-Commit" state to make the protocol non-blocking under coordinator failure.
- **Analysis:** Simulate a network partition where the Coordinator and one Participant are separated from the rest.
- **Metric:** Show that 3PC allows the remaining "functional" partition to reach a decision without waiting for the coordinator to recover.

### 35. Distributed Log Recovery Manager: "Post-Crash Analysis"

- **Dataset:** A "Dirty" transaction log containing START, PREPARE, COMMIT, and ABORT entries with missing ends.
- **The Task:** Write a recovery script that reads the logs of 3 different sites.
- **Analysis:** Determine which transactions should be **Redone** and which should be **Undone** based on the 2PC rules in Özsu and Valduriez.
- **Deliverable:** A "Clean" state of the database after the recovery script finishes.

### 36. Cooperative Termination Protocol: "Airline Booking"

- **Dataset:** Seats table.

- **The Task:** Implement a protocol where, if the coordinator fails, participants contact *each other* to decide the outcome.
- **Analysis:** If any participant has received a "Commit" or "Abort," the group decides. If all are in the "Ready" state, they must remain blocked.
- **Metric:** Number of "Peer-to-Peer" messages exchanged before a decision is reached.

### 37. Quorum-Based Commit Protocol: "Replicated File System"

- **Dataset:** 3 replicas of a File_Metadata table.
- **The Task:** Implement a commit protocol where a transaction commits if a **Quorum** (majority) of participants are ready.
- **Analysis:** Simulate a "Slow Link" to one site.
- **Deliverable:** Prove that the system remains available (commits successfully) even when 1 out of 3 sites is unreachable.

### 38. Time-Based Leases for Reliability: "IoT Sensor Data"

- **Dataset:** A stream of Sensor_Readings.
- **The Task:** Instead of waiting indefinitely for a commit, implement **Leases** (time-outs). If a participant doesn't hear from the coordinator in $X$ milliseconds, it defaults to Abort.
- **Analysis:** Fine-tune the "Lease Duration." If it's too short, transactions abort unnecessarily; if too long, the system blocks.
- **Metric:** Success Rate vs. Lease Duration (ms).

### 39. Simplified Byzantine Fault Tolerance: "Distributed Ledger"

- **Dataset:** A set of Transaction_Requests.
- **The Task:** Implement a system where 1 of the 4 sites is "Malicious" (randomly sends ABORT instead of COMMIT).
- **Analysis:** Use the $3n + 1$ rule to ensure the remaining 3 sites reach the correct consensus.
- **Deliverable:** A demo showing the system ignoring the "traitor" node and committing the correct data.

### 40. Log Pruning and Checkpointing: "High-Frequency Trading"

- **Dataset:** 100,000 transaction logs.
- **The Task:** Implement a **Global Checkpointing** algorithm.
- **Analysis:** Determine the "Safe Point" where logs can be deleted across all distributed sites without risking recovery.
- **Metric:** Disk space saved after each checkpointing cycle.

---

For these reliability projects, I suggest that students use **Python's multiprocessing library** or **Node.js worker_threads** to simulate multiple "sites" on a single laptop while accurately modeling communication delays and process crashes.

---

Grading : Category 4

| Criteria | Excellent (90-100%) | Satisfactory (70-89%) | Developing (<70%) |
|---|---|---|---|
| **State Accuracy** | Correctly handles all states (Ready, Commit, Abort). | Missing one state transition logic. | Incomplete state machine. |
| **Failure Handling** | Recovers correctly from a simulated crash. | Recovers but requires manual intervention. | Data is corrupted after crash. |
| **Log Management** | Logs are formatted correctly for recovery. | Basic logging with some missing info. | No logging (in-memory only). |
| **Textbook Alignment** | References specific 2PC/3PC rules from Özsu. | General understanding of reliability. | Misunderstands the commit flow. |

# Category 5: Replication Management

### 41. Read-One-Write-All (ROWA) Protocol: "User Profiles"

- **Dataset:** User_Profiles (UserID, Bio, AvatarURL, LastLogin) replicated across 3 nodes.
- **The Task:** Implement the strictest form of replication. A read can go to any node, but a write must update *every* node successfully.
- **Analysis:** Simulate a single node failure during a write operation.
- **Metric:** Show that the system prioritizes **Consistency** over **Availability** (i.e., the write is rejected if even one node is down).

### 42. ROWA-Available (ROWA-A): "Warehouse Inventory"

- **Dataset:** Stock_Levels (SKU, Quantity, WarehouseID).
- **The Task:** Improve upon ROWA. If a node is down, the write proceeds to all *available* nodes. When the failed node recovers, it must "catch up."
- **Analysis:** Implement a "Recovery Log" for the downed node to read upon reboot.
- **Deliverable:** A demo showing a "Stale Read" being prevented by the recovery mechanism.

### 43. Quorum-Based Voting (Static): "Global Configuration"

- **Dataset:** System_Settings (Key, Value, Version). Total sites $N = 5$.
- **The Task:** Implement the Quorum requirement where $V_r + V_w > V$ and $V_w > \frac{V}{2}$.
- **Analysis:** Use LaTeX to define your quorums: $V_r = 2$ and $V_w = 4$ for $V = 5$.
- **Metric:** Test the "Read-Your-Writes" consistency. If you write to a quorum of 4, can you always find the latest version by reading from any 2?

### 44. Primary Copy Replication: "Product Catalog"

- **Dataset:** Products (PID, Name, Price, Description).
- **The Task:** Designate Site 1 as the **Primary**. All updates go to Site 1, which then asynchronously (or synchronously) propagates to Sites 2 and 3 (Secondaries).
- **Analysis:** Simulate a "Primary Election" if Site 1 crashes.
- **Deliverable:** A log showing the handover of "Primary" status to Site 2 and the redirection of write traffic.

### 45. Vector Clocks for Conflict Detection: "Distributed Doc Edit"

- **Dataset:** Document_Fragments (DocID, Content, VectorClock).
- **The Task:** Use Vector Clocks (e.g., $[S_1: 1, S_2: 4]$) to track causality between updates in a multi-master setup.
- **Analysis:** Create a "Branch" (conflict) where two sites update the same line of text without knowing about each other.
- **Metric:** Correct identification of "Concurrent Updates" vs. "Causal Updates."

### 46. Anti-Entropy Gossip Protocol: "Fleet GPS Data"

- **Dataset:** Vehicle_Locations (VehicleID, Lat, Long, Timestamp).
- **The Task:** Nodes do not update each other immediately. Instead, every 2 seconds, a node picks a random "peer" and syncs missing data.
- **Analysis:** Measure the **Convergence Time**. How long does it take for a new coordinate at Site A to reach Site E?
- **Deliverable:** A heat map or table showing "Data Consistency %" over time.

### 47. Lazy Replication Inconsistency Window: "Social Likes"

- **Dataset:** Post_Likes (PostID, LikeCount).
- **The Task:** Implement "Lazy" replication where updates are sent in batches every 5 seconds.
- **Analysis:** From a user's perspective, how "stale" is the data?
- **Metric:** Measure the "Inconsistency Window" (the time difference between the Primary update and the last Secondary update).

### 48. Master-Master with Last-Write-Wins (LWW): "Customer Address"

- **Dataset:** Customer_Info (CID, Address, Phone, Update_Timestamp).
- **The Task:** Allow writes to any node. Use the system clock to resolve conflicts (LWW).
- **Analysis:** Discuss the dangers of "Clock Skew." What if Site A's clock is 2 seconds ahead of Site B's?
- **Deliverable:** A simulation showing a "newer" update from a "slower" clock being incorrectly discarded.

### 49. Eager vs. Lazy Comparison: "Transaction Ledger"

- **Dataset:** 10,000 Bank_Transactions.
- **The Task:** Build two systems—one using Eager (synchronous) replication and one using Lazy (asynchronous).
- **Analysis:** Compare **Write Latency** vs. **Data Durability**.
- **Metric:** A bar chart showing the throughput (transactions per second) for both methods.

### 50. Partition Tolerance (Split-Brain) Test: "Key-Value Store"

- **Dataset:** A simple Key-Value table.
- **The Task:** Simulate a "Network Partition" where Sites $\{1, 2\}$ cannot talk to $\{3, 4, 5\}$.
- **Analysis:** Apply the **Majority Rule**. The minority partition must become "Read Only" to prevent divergent histories.
- **Deliverable:** Prove that when the network heals (merges), the "Majority" data survives and the "Minority" updates are rolled back or synchronized.

## Grading : Category 5

| Criteria | Excellent | Satisfactory | Developing |
|---|---|---|---|
| **Consistency Model** | Perfectly maintains the chosen model (ROWA, Quorum, etc.). | Occasional "Stale Reads" in strict models. | No clear consistency logic. |

| | | | |
|---|---|---|---|
| **Conflict Handling** | Robust resolution (Vector Clocks or LWW). | Detects conflicts but can't resolve them. | Overwrites data blindly. |
| **Failure Simulation** | Clear "Node Down/Up" testing. | Minimal testing of failure states. | Assumes a perfect network. |
| **Performance Metrics** | Analysis of Latency vs. Consistency trade-offs. | Basic "it works" description. | No performance data. |

---

# Category 6: NoSQL & Key-Value Stores

### 51. Consistent Hashing Ring: "Cloud Storage Load Balancer"

- **Dataset:** A list of 10,000 File_IDs (UUIDs) and 5 Storage_Nodes.
- **The Task:** Implement a consistent hashing algorithm where keys and nodes are mapped onto a $2^{32}$ circle.
- **Analysis:** Simulate adding a 6th node.
- **Metric:** Calculate the "Churn Rate"—what percentage of keys had to be moved to the new node? Compare this against a simple Hash(key) % N approach.

### 52. Sharding by Key Range: "Global User Directory"

- **Dataset:** Users table (Username, Email, Country).
- **The Task:** Implement sharding based on alphabetical ranges of Username (e.g., Shard 1: A–G, Shard 2: H–N, etc.).
- **Analysis:** Provide a dataset where 70% of usernames start with 'S' or 'M'.
- **Deliverable:** A report on **Shard Hotspots** and a proposed "Re-sharding" logic to balance the load.

### 53. Schema-on-Read Document Store: "IMDb Movie Data"

- **Dataset:** A 50MB JSON file of Movies where some records have Director, others have Director_List, and some have no director field at all.
- **The Task:** Build a simple query engine that allows searching across these heterogeneous JSON objects.
- **Analysis:** Compare the flexibility of this "NoSQL" approach against a strict SQL schema that would require multiple NULL columns or join tables.
- **Metric:** Time taken to execute a "Full Text Search" on a nested field.

### 54. Leaderless Replication (Sloppy Quorum): "IoT Sensor Hub"

- **Dataset:** Temp_Readings (SensorID, Value, Timestamp) arriving at 100/second.
- **The Task:** Implement a Dynamo-style leaderless system. A write is successful if any $W$ nodes out of $N$ acknowledge it.
- **Analysis:** Simulate a "Network Glitch" where Node 1 is temporarily partitioned. Implement **Hinted Handoff** (Node 2 holds the data for Node 1).
- **Deliverable:** A trace showing the data being delivered to Node 1 once the partition heals.

### 55. LSM-Tree Compaction Simulator: "High-Velocity Logs"

- **Dataset:** A continuous stream of Log_Entries (Timestamp, ErrorLevel, Message).
- **The Task:** Instead of updating a B-Tree, write incoming data to an in-memory "MemTable." When full, flush to a sorted "SSTable" on disk.
- **Analysis:** Implement a "Background Merger" that combines two SSTables and removes duplicate keys (Compaction).
- **Metric:** Write Latency vs. Read Latency (Read latency increases as the number of un-compacted SSTables grows).

### 56. Distributed LRU Cache: "URL Shortener"

- **Dataset:** 100,000 ShortURL -> LongURL mappings.
- **The Task:** Implement a Least Recently Used (LRU) cache distributed across 3 nodes using a "Gossip" protocol to invalidate entries.

- **Analysis:** When a URL is updated on Node A, Node B and C must eventually evict their stale cached version.
- **Deliverable:** A "Hit/Miss Ratio" report for a workload with a "Long Tail" distribution (a few URLs are very popular).

### 57. Polyglot Persistence Bridge: "Orders & Reviews"

- **Dataset:** Orders in a relational CSV (PostgreSQL style) and Product_Reviews in a JSON file (MongoDB style).
- **The Task:** Write a "Mediation Layer" that performs a distributed join between the two.
- **Analysis:** Analyze the "N+1 Query Problem." Is it faster to fetch all reviews first or fetch them one-by-one as you iterate through orders?
- **Metric:** Total execution time for a "Show top 10 products with their latest 5 reviews" query.

### 58. Geo-Partitioning in NoSQL: "Global Ride-Hailing"

- **Dataset:** Driver_Locations (DriverID, City, Lat, Long).
- **The Task:** Automatically route "Write" requests to the server closest to the driver's city (e.g., Paris drivers write to the EU-West shard).
- **Analysis:** What happens if a driver moves from Paris to London?
- **Deliverable:** A "Routing Logic" script that handles cross-shard migration of a driver’s record.

### 59. Local-First MapReduce: "Word Frequency Analysis"

- **Dataset:** 10 large text files (Project Gutenberg books) distributed across 5 folders (representing nodes).
- **The Task:** Implement a local "Map" (count words in each file) and a central "Reduce" (aggregate counts).
- **Analysis:** Discuss why moving the *computation* (the script) to the *data* is better than moving the *data* to a central server.
- **Metric:** Compare total network bytes used for "Local Map" vs. "Centralized Count."

### 60. Version-Based Conflict Resolution: "Shopping Cart"

- **Dataset:** Cart_Items (SessionID, ItemList, Version).
- **The Task:** Implement a multi-master shopping cart. If a user adds an item from their Phone and another from their Laptop simultaneously, the system must merge the carts.
- **Analysis:** Use "Causal Ordering" to ensure that if a user deletes an item, it stays deleted.
- **Deliverable:** A demonstration of a "Divergent Cart" being successfully merged into a single consistent list.

---

Grading : Category 6

| Criteria | Excellent (90-100%) | Satisfactory (70-89%) | Developing (<70%) |
|---|---|---|---|
| **Scalability Logic** | Successfully demonstrates horizontal growth (adding nodes). | Logic works but requires manual re-balancing. | System is effectively centralized. |
| **Data Model** | Appropriately uses JSON/KV pairs; understands "Schema-less" benefits. | Uses KV pairs but forces a rigid relational structure on them. | Misunderstands the NoSQL data model entirely. |
| **Trade-off Analysis** | Clearly explains the CAP Theorem choices made (e.g., why $W = 1$ was chosen). | Mentions CAP but cannot relate it to their specific project results. | No mention of consistency vs. availability trade-offs. |
| **Implementation** | Efficient handling of unstructured data. | Code works but is highly inefficient (e.g., $O(n^2)$ searches). | Code fails on large datasets. |

# Category 7: Peer-to-Peer (P2P) Data Management

### 61. Simple Chord DHT Implementation: "The Hash Ring"

- **Dataset:** A list of 1,000 Resource_IDs (hashed using SHA-1) and 50 Node_IDs.
- **The Task:** Implement the **Chord** Distributed Hash Table (DHT) protocol. Each node must maintain a "Finger Table" to route queries.
- **Analysis:** Prove the $O(log\ N)$ lookup property.
- **Metric:** Count the number of hops required to find a key as the number of nodes $(N)$ increases from 10 to 50.

### 62. Gnutella-style Flooding: "Unstructured Search"

- **Dataset:** A graph representing 100 nodes with random connections (edges). Each node holds a random list of 5 File_Names.
- **The Task:** Implement a search using **Flooding** with a **Time-to-Live (TTL)**.
- **Analysis:** Evaluate the "Search Coverage" vs. "Network Overhead."
- **Deliverable:** A graph showing how many files are found vs. the total number of messages sent for $TTL = 3, 5, and\ 7$.

### 63. Content Addressable Network (CAN): "2D Data Space"

- **Dataset:** 500 data points with $(x, y)$ coordinates (e.g., $x = Price, y = Rating$).
- **The Task:** Implement a 2D **CAN overlay**. Nodes "own" a rectangular zone in the coordinate space.
- **Analysis:** Demonstrate how the coordinate space "splits" when a new node joins the network.
- **Metric:** Measure the average number of routing steps to reach a specific coordinate.

### 64. Distributed Inverted Index: "P2P Library Search"

- **Dataset:** 100 text documents (Short Stories).
- **The Task:** Create a P2P search engine. Each peer indexes a subset of documents and stores "Keyword -> DocID" mappings in a DHT.
- **Analysis:** Perform a multi-keyword search (e.g., "Distributed" AND "Database").
- **Deliverable:** A trace showing which peers were contacted to resolve the boolean "AND" query.

### 65. Handling Churn (Node Join/Leave): "The Unstable Network"

- **Dataset:** Use the Chord setup from Project 61.
- **The Task:** Create a simulation where 20% of nodes "leave" the network simultaneously (Churn).
- **Analysis:** Implement **Successor Lists** to ensure the ring doesn't break.
- **Metric:** Measure the "Lookup Success Rate" immediately after the churn event vs. after the network stabilizes (fixes pointers).

### 66. BitTorrent-style Chunking: "Large File Distribution"

- **Dataset:** A 10MB "Master File" split into 256KB chunks.
- **The Task:** Implement a "Rarest-First" downloading strategy. 10 peers start with different random chunks of the file.
- **Analysis:** Show how peers trade chunks to eventually get the full file.
- **Metric:** Total time for all 10 peers to reach 100% completion compared to a "Random-First" strategy.

### 67. P2P Range Queries via Prefix Hash Trees: "Sensor Alerts"

- **Dataset:** Temperature_Data (Peers hold values from $-20$ to $50$).
- **The Task:** Standard DHTs are bad for range queries (e.g., "Find all values between 20 and 30"). Implement a **Prefix Hash Tree (PHT)** over a DHT to solve this.
- **Analysis:** Compare the number of messages for a range query vs. individual point lookups.
- **Deliverable:** A report on the efficiency of the PHT for narrow vs. wide ranges.

### 68. Gossip-based Membership Protocol: "Who is Online?"

- **Dataset:** A list of 100 IP addresses (simulated).
- **The Task:** Instead of a central registry, peers maintain a "Partial View" of the network. They periodically swap lists of known "Live" peers with a neighbor.
- **Analysis:** Introduce a "Dead Node." How long does it take for the entire network to realize that node is gone?
- **Metric:** Percentage of peers with an accurate "Live List" over time intervals.

### 69. EigenTrust Lite (Reputation): "Filtering Malicious Peers"

- **Dataset:** Transaction history of peers sharing files.
- **The Task:** Implement a simple reputation system where Peer A rates Peer B after a download.
- **Analysis:** Simulate 5 "Malicious" peers who send "Garbage Data" (corrupted chunks).
- **Metric:** Show the "Success Rate" of downloads over time as the system learns to ignore low-reputation peers.

### 70. Sybil Attack Simulator: "Identity Inflation"

- **Dataset:** A P2P voting system (e.g., "Should we delete this file?").
- **The Task:** Demonstrate a **Sybil Attack** where one user creates 50 fake identities (nodes) to swing the vote.
- **Analysis:** Implement a "Proof of Work" or "Trusted Bootstrap" to mitigate the attack.
- **Deliverable:** A "Before vs. After" comparison of the voting results with and without the Sybil protection.

---

Grading : Category 7

| Criteria | Excellent | Satisfactory | Developing |
|---|---|---|---|
| **Routing Logic** | Correctly implements DHT or Flooding rules; handles multi-hop paths. | Basic routing works, but may loop or fail on large networks. | Routing is broken or effectively centralized. |
| **Churn Resilience** | System recovers and fixes pointers after nodes leave. | System works but crashes if the "wrong" node leaves. | No handling of node failures. |
| **Analytical Metrics** | Clear data on Hops, Latency, or Message Overhead. | Basic count of successful searches. | No quantitative data provided. |
| **Implementation** | Efficient message passing (simulated or real). | Code is very slow; handles only small $N$. | Code fails to simulate peer independence. |

P2P projects can be visually stunning. If students use **Python with NetworkX**, they can generate graphs showing the actual network topology and the paths their queries take. This makes for an impressive final presentation.

---

# Category 8: Web Data & XML/JSON (Data Integration)

### 71. Distributed XPath Evaluator: "Digital Library"

- **Dataset:** A large XML file of Book_Records (Title, Author, Year, Chapters, Citations).
- **The Task:** Implement a simplified XPath engine that evaluates queries (e.g., /library/book[year>2020]/title) across three distributed XML fragments.
- **Analysis:** Compare the performance of "Fragment-First" filtering vs. "Fetch-and-Filter" at the coordinator.
- **Deliverable:** A script that returns the correct node-set while minimizing the number of XML nodes transmitted over the network.

### 72. XML Horizontal vs. Vertical Fragmentation: "Electronic Health Records"

- **Dataset:** Patient_History.xml containing nested tags for Bio, Visits, and Prescriptions.
- **The Task:** Implement two versions: (1) Horizontal (splitting by Patient ID) and (2) Vertical (splitting by sub-trees like <Prescriptions>).
- **Analysis:** Determine which fragmentation style is more efficient for a query like "List all patients who were prescribed Aspirin."
- **Metric:** Measure the "Reconstruction Cost" (joining XML trees) for vertical fragments.

### 73. JSON Schema Matcher: "E-Commerce Aggregator"

- **Dataset:** Two JSON feeds from different "Store" APIs. Store A uses { "prod_name": "...", "price": 10 }, Store B uses { "item": "...", "cost": { "amt": 10, "cur": "USD" } }.
- **The Task:** Build a **Mediator** that uses a "Global Schema" to map these two heterogeneous sources into a single unified JSON output.
- **Analysis:** Document the mapping rules (GAV - Global As View vs. LAV - Local As View).
- **Deliverable:** A unified API endpoint that queries both "stores" and merges the results in real-time.

### 74. Distributed Web Indexer: "Topic-Specific Search"

- **Dataset:** A collection of 500 HTML pages spread across 3 local folders.
- **The Task:** Implement a mini-MapReduce to build a **Distributed Inverted Index**. One node indexes "A-M" keywords, another "N-Z".
- **Analysis:** Measure the time taken to build the index in parallel vs. a single-threaded local indexer.
- **Metric:** Total indexing time and search latency for a 2-word query.

### 75. RESTful Data Integration Wrapper: "Travel Planner"

- **Dataset:** A "Flight API" (returns JSON) and a "Weather API" (returns XML).
- **The Task:** Build a "Wrapper" for each source that translates their native format into a standard internal object.
- **Analysis:** Implement a "Join" in the mediator that only shows flights to cities where the weather is "Sunny."
- **Deliverable:** A dashboard showing the combined data retrieved from two physically distinct web services.

### 76. XQuery Decomposition: "Global Census Analysis"

- **Dataset:** Census_Data.xml fragmented by Country.
- **The Task:** Write an XQuery that performs an aggregation (e.g., sum(//population)) across all fragments.
- **Analysis:** Show how the query is decomposed into sub-queries sent to each country fragment.
- **Metric:** Communication overhead (bytes) vs. the complexity of the XQuery path.

### 77. JSON-LD Linker: "Open Data Knowledge Graph"

- **Dataset:** Snippets from DBpedia (JSON-LD) and a local "University Research" JSON file.
- **The Task:** Use the @context and @id tags in JSON-LD to link local researchers to their global DBpedia entries.
- **Analysis:** Evaluate the "Linkage Accuracy." How many local records were successfully "disambiguated" and linked to the web of data?
- **Deliverable:** A graph visualization (or list) showing the newly formed links between local and global data.

### 78. Distributed ETL for Web Logs: "Cybersecurity Monitor"

- **Dataset:** 100MB of raw Apache Web Logs (text format) across 4 sites.
- **The Task:** Build a distributed **Extract-Transform-Load (ETL)** pipeline. Extract IP addresses, transform them into Geo-locations, and load them into a central JSON store.
- **Analysis:** Implement a "De-duplication" logic so that the same IP appearing at two sites is only counted once.
- **Metric:** Throughput (Rows/Second) as more nodes are added to the ETL process.

### 79. Temporal Versioning in Distributed JSON: "Wiki Edits"

- **Dataset:** A series of JSON documents representing edits to a "Project Specification."
- **The Task:** Store versions of the document across two nodes. Implement a "Time-Travel Query" (e.g., "What did the JSON look like at 10:00 AM yesterday?").
- **Analysis:** Analyze the storage trade-off between "Full Snapshots" and "Delta Encoding" (storing only changes).
- **Deliverable:** A report comparing the storage size and retrieval time for both methods.

### 80. Distributed K-Anonymity for JSON: "Health Surveys"

- **Dataset:** A JSON dataset of User_Surveys (Age, ZipCode, Disease).
- **The Task:** Implement a distributed algorithm that ensures no individual can be identified by their "Quasi-identifiers" (Age + ZipCode).
- **Analysis:** Nodes must coordinate to ensure that across the entire distributed system, there are at least $k = 3$ people with the same quasi-identifiers.
- **Metric:** "Information Loss" (how much you had to generalize the data) to achieve $k$-anonymity.

---

Grading : Category 8

| Criteria | Excellent (90-100%) | Satisfactory (70-89%) | Developing (<70%) |
|---|---|---|---|
| **Data Mapping** | Flawless transformation between XML/JSON or Schema A/B. | Minor errors in mapping nested attributes. | Fails to handle semi-structured nesting. |
| **Mediator Logic** | Correctly implements GAV/LAV or Wrapper patterns. | Functional but hard-coded for specific files. | Logic is effectively centralized. |
| **Query Efficiency** | Demonstrates query pushdown or minimal data transfer. | Fetches entire files then filters (inefficient). | No optimization for web data retrieval. |
| **Documentation** | Clear mapping diagrams (Mediator/Wrapper). | Basic description of files. | No schema documentation. |

For **Project #73 (Schema Matching)**, provide the students with "Messy" data. For example, dates in different formats (DD-MM-YYYY vs YYYY/MM/DD). This forces them to deal with the real-world headache of Web Data Integration.

---

# Category 9: Distributed Object Management

### 81. Horizontal Object Fragmentation: "Social Media Profiles"

- **Dataset:** A set of User objects with attributes: OID, Username, Bio, Region, and Interests[] (an array/collection).
- **The Task:** Fragment the object collection horizontally based on the Region attribute.
- **Analysis:** Unlike relational fragmentation, you must ensure that **Object Identity (OID)** remains consistent across sites. If a user moves regions, does the OID change?
- **Deliverable:** A system that retrieves a User object by OID regardless of which regional site it currently resides on.

### 82. Vertical Object Fragmentation: "RPG Game Characters"

- **Dataset:** Complex Character objects containing BaseStats (Level, HP), Inventory (List of Item Objects), and SkillTree (Nested JSON/Object).
- **The Task:** Vertically fragment the objects. Store BaseStats on a "Fast-Access" node and Inventory/SkillTree on a "High-Storage" node.
- **Analysis:** Measure the overhead of **Object Reconstruction**. When a player opens their menu, the system must join these fragments back into a single object instance.
- **Metric:** Time to "rehydrate" an object from 2 sites vs. 1 site.

### 83. Distributed OID Generator: "Unique Identifier Service"

- **Dataset:** A simulation of 4 sites creating 10,000 objects each per second.
- **The Task:** Implement a distributed **Object Identity (OID)** generation scheme that guarantees global uniqueness without a central coordinator.
- **Analysis:** Compare **UUID/GUID** (random/time-based) against a **Hi-Lo Algorithm** (where sites request "blocks" of IDs).
- **Deliverable:** A report on the probability of ID collisions and the "ID-Vending" latency for each method.

### 84. Distributed Object Caching: "Product Metadata"

- **Dataset:** 5,000 Product objects with deeply nested attributes (Specifications, Reviews).
- **The Task:** Implement a "Local-Remote" cache. Site A frequently accesses Product X; it should cache it locally.
- **Analysis:** Implement a **Cache Invalidation** protocol. If Site B updates Product X, Site A must be notified to drop its stale cache.
- **Metric:** "Cache Hit Ratio" vs. "Invalidation Traffic" (number of messages).

### 85. The "N+1 Problem" in Distributed ORM: "Author-Books"

- **Dataset:** Author objects and Book objects linked by OIDs.
- **The Task:** Simulate a query: "Get all Authors in the UK, and for each, get their 50 latest Books."
- **Analysis:** Compare **Lazy Loading** (one network call per author to get books) vs. **Eager Loading** (one giant join/batch call).
- **Deliverable:** A graph showing how network latency $(50ms+)$ makes Lazy Loading unusable in distributed environments.

### 86. Distributed Garbage Collection (DGC): "Temporary Session Objects"

- **Dataset:** A graph of objects where Site A's objects hold references to Site B's objects.
- **The Task:** Implement a simplified **Reference Counting** DGC.
- **Analysis:** Handle the "Liveness" problem. If Site A crashes, Site B must eventually realize the references are gone and clean up its memory.
- **Metric:** Memory leaked (objects not deleted) vs. false deletions (objects deleted too early).

### 87. Navigational vs. Declarative Queries: "Corporate Hierarchy"

- **Dataset:** A tree of Employee objects where each has a Supervisor OID reference.
- **The Task:** Implement two search methods: (1) **Navigational** (programmatically following OIDs from Site to Site) and (2) **Declarative** (an OQL-like query SELECT e FROM Employees WHERE e.Supervisor.Region = 'Asia').
- **Analysis:** Which is faster for a deep 5-level hierarchy?
- **Deliverable:** A trace of "Site-Hops" for both methods.

### 88. Versioning Distributed Objects: "Collaborative Design"

- **Dataset:** CAD_Model objects (PartID, Geometry, Version).
- **The Task:** Allow two sites to check out the same object. If both "Check In" different versions, implement a **Conflict Resolution** strategy (e.g., branching or timestamp-based).
- **Analysis:** How do you store "Object Deltas" across sites to save space?
- **Metric:** Storage used for 10 versions using "Full Snapshot" vs. "Delta Storage."

### 89. Distributed Inheritance Handling: "Vehicle Fleet"

- **Dataset:** A base class Vehicle (at Site 0), and subclasses Truck (Site 1) and ElectricCar (Site 2).
- **The Task:** Implement a query on the superclass Vehicle that must aggregate data from all subclass sites.
- **Analysis:** Discuss the **Schema Evolution** problem: if you add an attribute to Vehicle, how do you propagate that change to the distributed subclasses?
- **Deliverable:** A script that correctly performs a "Polymorphic Search" across all sites.

### 90. Object Serialization Comparison: "Cross-Platform Sync"

- **Dataset:** A 1,000-object array with mixed data types (String, Int, Float, Nested List).
- **The Task:** Measure the cost of moving objects between sites using three formats: **JSON**, **XML**, and **Protocol Buffers (Binary)**.
- **Analysis:** Analyze the trade-off between "Human-Readability" and "Network Throughput."
- **Metric:** Serialization time, Deserialization time, and Total Byte Size for the same object.

---

Grading : Category 9

| Criteria | Excellent (90-100%) | Satisfactory (70-89%) | Developing (<70%) |
|---|---|---|---|
| **OID Management** | Flawless handling of object identity across sites. | OIDs work but are inefficient or prone to collisions. | Fails to maintain object identity. |
| **Complexity Handling** | Correctly manages nested objects or class hierarchies. | Handles flat objects well but struggles with nesting. | Only handles primitive data types. |
| **Network Awareness** | Clearly accounts for the cost of "Object Rehydration." | Functional but assumes zero-latency object fetching. | No consideration for network costs. |
| **Analysis** | Deep dive into "Serialization" or "Garbage Collection" logic. | Basic explanation of the code. | No theoretical analysis. |

These projects are perfect for students who enjoy **Software Engineering** and **OOP**. If they are comfortable with Python, they can use the pickle or marshmallow libraries to handle the object-to-data transformations easily.

---

# Category 10: Performance Benchmarking

### 91. TPC-C "New Order" Simulator: "Retail Stress Test"

- **Dataset:** A simplified TPC-C schema: Warehouse, District, Customer, Item, Stock, and Orders. (Approx. 50,000 rows).
- **The Task:** Implement the "New Order" transaction across two sites (one for Warehouse, one for Customer).
- **Analysis:** Measure the **Throughput** (Transactions Per Second - TPS) as you increase the number of concurrent users from 1 to 20.
- **Metric:** A line graph showing the "Saturation Point" where adding more users no longer increases throughput.

### 92. Read-Heavy vs. Write-Heavy Workloads: "News Portal"

- **Dataset:** Articles (ID, Content) and Comments (ID, ArticleID, Text).
- **The Task:** Create two test scripts. Script A: 95% Reads / 5% Writes. Script B: 50% Reads / 50% Writes.
- **Analysis:** Analyze how **Replication** (from Category 5) helps Script A but hurts Script B due to update overhead.
- **Deliverable:** A comparison table showing the "Average Latency" for both workloads on a 3-node replicated cluster.

### 93. Network Latency Impact Study: "Trans-Atlantic DB"

- **Dataset:** Financial_Transactions (10,000 records).
- **The Task:** Use a tool (like tc on Linux or a simple sleep() in code) to simulate network delays: 1ms (Local), 50ms (Regional), and 250ms (Global).
- **Analysis:** Run a 2-Phase Commit (2PC) transaction across these simulated latencies.
- **Metric:** Calculate the **Cost of Coordination**—how much of the total transaction time is "waiting for the network" vs. "doing work."

### 94. Horizontal Scaling Efficiency: "Sharding Gains"

- **Dataset:** 1 million User_Logs.
- **The Task:** Benchmark the time to run an aggregation query (COUNT grouped by UserID) on 1 node, 2 nodes, and 4 nodes.
- **Analysis:** Calculate the **Linear Speedup Ratio**. If 4 nodes take 1/4th the time of 1 node, speedup is 1.0 (perfect).
- **Metric:** Use the formula: $Speedup = \frac{T_1}{T_n}$ where $T_1$ is execution time on one node and $T_n$ is time on $n$ nodes.

### 95. Distributed Join Algorithm Shootout: "Orders & Items"

- **Dataset:** Orders (Site A, 10k rows) and OrderItems (Site B, 100k rows).
- **The Task:** Benchmark two join strategies: (1) **Ship-Whole-Table** (moving all of Site B to A) and (2) **Semi-Join** (moving unique IDs).
- **Analysis:** Find the "Breakeven Point." At what ratio of OrderItems to Orders does a Semi-Join become slower than a simple Join?
- **Deliverable:** A report defining the "Selectivity" threshold where the strategy should switch.

### 96. The "Hot Shard" Problem: "Viral Tweet Analysis"

- **Dataset:** Tweets partitioned by AuthorID.
- **The Task:** Create a "Skewed Workload" where one AuthorID (a celebrity) gets 80% of the traffic, while 1,000 others get 20%.
- **Analysis:** Measure the CPU utilization and Queue Depth of the "Hot Node" vs. the "Cold Nodes."
- **Metric:** Plot the "Load Imbalance" (Difference in latency between the fastest and slowest node).

### 97. YCSB (Yahoo! Cloud Serving Benchmark) Lite: "KV Store Performance"

- **Dataset:** 100,000 Key-Value pairs (1KB each).
- **The Task:** Implement the 4 basic YCSB workloads: (A) 50/50 R/W, (B) 95/5 R/W, (C) 100% Read, and (D) Read-Latest.
- **Analysis:** Evaluate how a **Distributed Hash Table (DHT)** handles these workloads compared to a **Centralized Index**.
- **Deliverable:** A "Performance Profile" for your DB implementation under each YCSB category.

### 98. Concurrency Control Overhead: "Locking vs. Timestamps"

- **Dataset:** Inventory table with 1,000 items.
- **The Task:** Compare the performance of **2-Phase Locking** (Category 3) against **Timestamp Ordering**.
- **Analysis:** Increase "Conflict Density" (multiple users trying to buy the *same* item).
- **Metric:** Measure the "Abort Rate." At what point do restarts (Timestamps) become more expensive than waiting (Locking)?

### 99. Recovery Time Objective (RTO) Benchmark: "Disaster Recovery"

- **Dataset:** 1GB of transaction logs and a 500MB database snapshot.
- **The Task:** Artificially "crash" a node and measure how long it takes for the **Recovery Manager** (Category 4) to bring it back to a consistent state.
- **Analysis:** How does the frequency of **Checkpointing** affect recovery time?
- **Metric:** Plot Recovery Time vs. Checkpoint Interval (e.g., every 1 min vs. every 10 mins).

### 100. Multi-tenant Isolation Performance: "SaaS Database"

- **Dataset:** Data for 10 "Tenants" (Users) sharing the same distributed cluster.
- **The Task:** Run a heavy "Reporting Query" for Tenant A while Tenant B is doing high-speed "Inserts."
- **Analysis:** Measure "Noisy Neighbor" interference. How much does Tenant A's query slow down Tenant B?
- **Deliverable:** A report on **Quality of Service (QoS)**, showing the latency variance for Tenant B during Tenant A's peak usage.

Grading : Category 10

| Criteria | Excellent (90-100%) | Satisfactory (70-89%) | Developing (<70%) |
|---|---|---|---|
| **Methodology** | Controls variables strictly; uses multiple runs to average results. | Runs tests once; minor external noise in data. | Haphazard testing; no clear baseline. |
| **Visual Analysis** | Clear, professional charts (Line/Bar/Heatmaps) with labeled axes. | Basic Excel charts; lacks depth in visualization. | No charts; just a wall of numbers. |
| **Statistical Rigor** | Uses $Mean$, $Median$, and P99 (Tail Latency) correctly. | Only reports the "Average" (which hides outliers). | No statistical analysis. |
| **Textbook Link** | Relates results back to Özsu's cost models ($Cost = IO + CPU + Comm$). | Mentions the book but doesn't use the formulas. | No theoretical connection. |

# Category 11: Security and Privacy

### 101. Distributed Role-Based Access Control (RBAC): "Global HR Portal"

- **Dataset:** An Employees table and a Roles_Permissions table (e.g., Admin, Manager, Intern).
- **The Task:** Implement a distributed RBAC system where a user logs in at Site A, but their permissions must be enforced when they query a fragment at Site B.
- **Analysis:** Handle the "Revocation Propagation" problem. If a manager's "Write" permission is revoked at the central site, how long does it take for Site B to stop accepting their updates?
- **Deliverable:** A script that successfully blocks an unauthorized cross-site JOIN query based on the user's token.

### 102. Distributed $k$-Anonymity: "Medical Research Database"

- **Dataset:** Patient_Records (Age, Gender, ZipCode, Disease).
- **The Task:** Implement a distributed algorithm where two sites want to share data for research without revealing identities.
- **Analysis:** Ensure that for every record, there are at least $k = 5$ other records with the same "Quasi-identifiers" (Age, Gender, ZipCode) across the *entire* distributed set.
- **Metric:** Measure "Information Loss"—how much did you have to broaden age ranges (e.g., "20-30" instead of "25") to achieve $k$-anonymity?

### 103. Shamir's Secret Sharing for DB Credentials: "The Vault"

- **Dataset:** A 256-bit AES Master Key used to encrypt a distributed database.
- **The Task:** Implement a $(t, n)$ threshold scheme. Split the key into 5 "shares" $(n = 5)$ and distribute them to 5 different site admins.
- **Analysis:** Prove that any $t = 3$ admins can reconstruct the key to unlock the DB, but 2 admins cannot.
- **Deliverable:** A Python/Java implementation of the polynomial interpolation required to recover the secret.

### 104. Secure Multi-party Computation (SMPC): "Salary Survey"

- **Dataset:** 4 Departments (Sites), each with a private Salary_Total.
- **The Task:** Calculate the **Global Average Salary** across all departments *without* any department revealing its specific total to the others or a central server.
- **Analysis:** Use a "Secure Sum" protocol (where each site adds a random number, passes it along, and subtracts it later).
- **Metric:** Prove the mathematical correctness: $\sum(Private\_Data) = Global\_Sum$ while maintaining privacy.

### 105. Merkle Tree Log Integrity: "Immutable Audit Trail"

- **Dataset:** Banking_Transactions (From, To, Amount).
- **The Task:** For every 100 transactions, generate a **Merkle Tree** (hash tree). Store the Root Hash at a "Trusted Third Party" site.
- **Analysis:** Simulate an "Insider Attack" where a database admin at Site B changes an amount from $\$100$ to $\$1{,}000$.
- **Deliverable:** A "Tamper Detector" script that compares the local tree against the Root Hash to identify exactly which transaction block was altered.

### 106. Privacy-Preserving Vertical Fragmentation: "PII Shield"

- **Dataset:** Customer_Data (Name, SSN, CreditCard, PurchaseHistory).
- **The Task:** Vertically fragment the table into a "Public" fragment (PurchaseHistory) and a "Secure" fragment (Name, SSN, CreditCard).
- **Analysis:** Encrypt the linking OID (Object ID) in the public fragment.
- **Metric:** Measure the "Query Latency" overhead when a legitimate user needs to perform a join to see a customer's name for a specific purchase.

### 107. Local Differential Privacy (LDP): "App Usage Statistics"

- **Dataset:** 10,000 simulated users reporting "App Category Used" (e.g., Social, Finance, Health).
- **The Task:** Before sending data to the central DB, each "client" (site) adds "Randomized Response" noise.
- **Analysis:** The central DB must be able to calculate the *global* trend (e.g., "30% use Finance apps") while having a 50% chance of being wrong about any *individual* user.
- **Metric:** Accuracy of the global estimate vs. the "Privacy Budget" ($\epsilon$).

### 108. Partially Homomorphic Encryption (PHE) Prototype: "Encrypted Cloud DB"

- **Dataset:** Product_Prices encrypted using the Paillier cryptosystem.
- **The Task:** Perform a SUM operation on the **encrypted** column without ever decrypting the individual prices.
- **Analysis:** Compare the time taken to sum 1,000 encrypted integers vs. 1,000 plaintext integers.
- **Deliverable:** A report on why this is useful for outsourcing data to untrusted cloud providers.

### 109. Distributed SQL Injection Firewall: "Query Guard"

- **Dataset:** A mix of "Clean" and "Malicious" SQL strings (e.g., '; DROP TABLE Users; --).
- **The Task:** Build a middleware layer that sits between the client and the distributed sites.
- **Analysis:** The middleware must parse the SQL and block any query that uses "Forbidden Patterns" or attempts to access fragments the user isn't assigned to.
- **Metric:** False Positive rate (clean queries accidentally blocked) vs. False Negative rate.

### 110. Replica Consistency Verification via Hash-Chains: "Anti-Desync"

- **Dataset:** 3 Replicas of a Product_Stock table.
- **The Task:** Every 10 minutes, each replica generates a hash-chain of its latest 100 updates.
- **Analysis:** The sites exchange these hashes. If Site 1's hash differs from Site 2's, a "State Divergence" has occurred.
- **Deliverable:** A "Reconciliation Tool" that identifies which specific transaction caused the replicas to get out of sync.

Grading : Category 11

| Criteria | Excellent (90-100%) | Satisfactory (70-89%) | Developing (<70%) |
|---|---|---|---|
| **Security Logic** | No "backdoors" or obvious leaks in the protocol. | System is secure but has high performance overhead. | Vulnerable to basic attacks (e.g., data leaks). |
| **Privacy Math** | Correct implementation of hashes, $k$-anonymity, or LDP. | Correct concepts but minor errors in noise/hash calculation. | Misunderstands the privacy algorithm. |
| **Distributed Context** | Effectively handles security across multiple sites. | Security is mostly centralized; limited distributed sync. | Effectively a single-site security model. |
| **Documentation** | Clear "Threat Model" (what is the system protected against?). | Basic list of security features. | No explanation of the security design. |

Security projects can be hard to "prove" work. I recommend requiring students to include a **"Hacker Mode"** in their demo: they must show a script that tries to steal or corrupt data and then show their system successfully stopping it.

---

# Category 12: Stream Processing & Real-Time Distributed DBs

### 111. Sliding Window Aggregator: "Crypto Ticker Monitor"

- **Dataset:** A real-time (or simulated) CSV stream of Bitcoin_Prices (Timestamp, Price, Volume).
- **The Task:** Implement a **Sliding Window** of 5 minutes that moves every 30 seconds.
- **Analysis:** Calculate the Moving Average and Volatility ($Standard\ Deviation$).
- **Metric:** Measure the "Processing Latency"—how many milliseconds pass between the arrival of a price and the updated window calculation?
- **Deliverable:** A live-updating console or dashboard showing the aggregate for the current window.

### 112. Distributed Watermark Tracker: "Log Delay Compensator"

- **Dataset:** Web_Server_Logs where some packets arrive "Out-of-Order" (e.g., a log from 10:01 arrives at 10:05).
- **The Task:** Implement **Watermarks** to handle event-time skew. The system must decide how long to wait for "late" data before closing a time window.
- **Analysis:** Compare "Strict Watermarks" (no data loss, high latency) vs. "Heuristic Watermarks" (some data loss, low latency).
- **Deliverable:** A report showing the "Data Completeness %" vs. "Wait Time" (ms).

### 113. Stream-to-Stream Join: "Ad-Click Attribution"

- **Dataset:** Two independent streams: Ad_Impressions (ImpressionID, UserID, AdID) and Ad_Clicks (ClickID, ImpressionID, Timestamp).
- **The Task:** Perform a real-time join on ImpressionID. Because clicks happen after impressions, the system must buffer impressions in memory.
- **Analysis:** Calculate the **Buffer Memory Usage**. How much RAM is needed to store unjoined impressions for a 1-hour window?
- **Metric:** The "Match Rate"—percentage of clicks successfully linked to an impression within a 10-second window.

### 114. Complex Event Processing (CEP): "Credit Card Fraud"

- **Dataset:** A stream of Bank_Transactions (CardID, Location, Amount, Timestamp).
- **The Task:** Detect a "Carding" pattern: three transactions of $< \$5.00$ followed by one transaction $> \$500.00$ within 2 minutes.
- **Analysis:** Implement a **State Machine** for each CardID to track the sequence of events.
- **Deliverable:** An "Alert Log" that triggers the moment the pattern is completed.

### 115. Backpressure Simulator: "IoT Sensor Overload"

- **Dataset:** 10 "Sensors" sending 1,000 packets/second to a single "Processor" node.
- **The Task:** Artificially slow down the Processor. Implement a **Backpressure** mechanism where the Processor signals the Sensors to slow down their sending rate.
- **Analysis:** Compare **Buffering** (storing extra packets in memory) vs. **Dropping** (discarding packets) vs. **Backpressure**.
- **Metric:** Total "Dropped Packets" and "Average Memory Usage" under each strategy.

### 116. Distributed State Checkpointing: "Game Score Sync"

- **Dataset:** A stream of Player_Actions (PlayerID, ActionType, Points).
- **The Task:** The "State" (total score) is stored in memory. Every 10 seconds, implement a **Snapshot** (Chandy-Lamport algorithm) to save the state across 3 nodes.
- **Analysis:** Simulate a node crash and measure the "Recovery Time."
- **Deliverable:** Prove that the scores after recovery match the scores before the crash without losing any processed events.

### 117. Exactly-Once Semantics (EOS): "Idempotent Payments"

- **Dataset:** A stream of Payment_Requests where some IDs are duplicated due to network retries.
- **The Task:** Ensure that each payment is processed **Exactly-Once** using a distributed "De-duplication Table."
- **Analysis:** Analyze the overhead of the "Write-Ahead Log" (WAL) required to guarantee EOS.
- **Metric:** Compare throughput ($TPS$) with EOS enabled vs. disabled ("At-Least-Once").

### 118. Top-K "Heavy Hitters" (Sketching): "Hashtag Trends"

- **Dataset:** A high-volume stream of Twitter_Hashtags (100,000+ entries).
- **The Task:** Use a **Count-Min Sketch** (a probabilistic data structure) to find the top 5 most frequent hashtags using only 1MB of memory.
- **Analysis:** Compare the "Approximate Count" from the sketch against the "Exact Count" from a giant hash map.
- **Deliverable:** A report on the "Error Margin" of the sketch as the data volume increases.

### 119. Continuous Range Query: "Ride-Hailing Proximity"

- **Dataset:** A stream of Driver_GPS coordinates $(x, y)$.
- **The Task:** Implement a **Continuous Query**: "Alert me whenever a driver enters a 5km radius of my location $(x_0, y_0)$."
- **Analysis:** Instead of re-running the query, update the result set incrementally as the stream flows.
- **Metric:** Time to update the "Nearby Drivers" list after a GPS coordinate change.

### 120. Multi-Stage Stream Pipeline: "Weather Alert System"

- **Dataset:** Weather_Sensors (Temp, Humidity, Pressure).
- **The Task:** Build a 3-stage pipeline: (1) **Clean** (remove nulls), (2) **Transform** (Celsius to Fahrenheit), (3) **Aggregate** (Average per County).
- **Analysis:** Distribute these 3 stages across 3 different nodes.
- **Metric:** Total **End-to-End Latency** $(T_{exit} - T_{entry})$. Which stage is the bottleneck?

Grading : Category 12

| Criteria | Excellent (90-100%) | Satisfactory (70-89%) | Developing (<70%) |
|---|---|---|---|
| **Windowing Logic** | Correct use of Event-Time vs. Processing-Time. | Basic tumbling windows; struggles with out-of-order data. | Misunderstands windowing concepts. |
| **State Management** | Efficiently manages and checkpoints distributed state. | State is stored but prone to loss on crashes. | System is stateless and cannot handle context. |
| **Latency Analysis** | Uses high-resolution timers; identifies bottlenecks. | Basic latency measurements provided. | No performance analysis. |
| **Robustness** | Handles backpressure or duplicates without crashing. | Crashes under high load or duplicate input. | Fails on simple data streams. |

Stream processing is best demonstrated with a **Live Visualization**. I suggest students use a simple library like Streamlit (Python) or a basic Chart.js (Web) to show the windows moving in real-time. It makes the "Data in Motion" concept click instantly.

---

# Category 13: Cloud-Native Databases (Docker & K8s)

### 121. Dockerized HA Cluster: "High Availability Retail"

- **Dataset:** Retail_Sales CSV.
- **The Task:** Deploy a 3-node PostgreSQL cluster using **Docker Compose**. Use **HAProxy** as a load balancer for read/write splitting.
- **Analysis:** Simulate a "Container Kill" on the Primary node.
- **Metric:** Measure the **Failover Time**—how many seconds of downtime occur before the Secondary takes over?

### 122. Kubernetes StatefulSet Scaling: "Dynamic User Store"

- **Dataset:** 10,000 User_Profiles.
- **The Task:** Deploy the database using a **Kubernetes StatefulSet**. Implement a script that increases the replica count from 3 to 6.
- **Analysis:** How does K8s handle the attachment of **Persistent Volumes (PVs)** to new pods?
- **Deliverable:** A log showing the sequential startup of pods and the data synchronization status.

### 123. Serverless DB Trigger: "Automatic Image Tagging"

- **Dataset:** Image_Metadata (ID, URL, Tags).
- **The Task:** Use a Serverless Function (e.g., AWS Lambda or local equivalent like OpenFaaS). When a row is inserted into Site A, the function must trigger an update on Site B.
- **Analysis:** Analyze the "Cold Start" latency of the serverless function.
- **Metric:** Average execution time of the trigger vs. the volume of incoming data.

### 124. Chaos Engineering for DBs: "The Resilient Reserve"

- **Dataset:** Flight_Reservations.
- **The Task:** Use a chaos tool (like Chaos Mesh) to randomly introduce network latency or pod failures in a K8s DB cluster.
- **Analysis:** Test the **Self-Healing** capabilities. Does the database maintain consistency during a network partition?
- **Deliverable:** A "Resilience Scorecard" for the distributed database.

### 125. Cloud-Native Backup/Restore: "Disaster Recovery S3"

- **Dataset:** Financial_Logs (500MB).
- **The Task:** Automate a distributed backup that chunks the DB and uploads it to an S3-compatible bucket (like MinIO).
- **Analysis:** Test a "Point-in-Time Recovery" (PITR).
- **Metric: Recovery Point Objective (RPO)**—how much data is lost if a crash occurs between backups?

### 126. Sidecar **Pattern** for Database Proxies: "The Security Sidecar"

- **Dataset:** User_Audit_Logs.

- **The Task:** Deploy a database pod in Kubernetes with a **Sidecar container** (e.g., using Envoy or a custom script) that intercepts all traffic to log queries before they hit the DB.
- **Analysis:** Measure the latency overhead introduced by the sidecar proxy.
- **Metric:** Compare "Direct Access" latency vs. "Sidecar-Proxied" latency.

### 127. Blue-Green Deployment for Schema Migration: "Zero-Downtime Update"

- **Dataset:** Product_Catalog_V1 (Old Schema) and Product_Catalog_V2 (New Schema).
- **The Task:** Use Kubernetes services to switch traffic from a "Blue" (v1) deployment to a "Green" (v2) deployment.
- **Analysis:** Implement a "Synchronization Bridge" that keeps both versions in sync during the transition.
- **Deliverable:** A report on the number of successful vs. failed transactions during the "Switch-over" window.

### 128. Cloud-Native Auto-Scaling (HPA): "Flash Sale Simulator"

- **Dataset:** A burst of 50,000 Order_Requests.
- **The Task:** Configure a **Horizontal Pod Autoscaler (HPA)** to spin up new database replicas based on CPU/Memory thresholds.
- **Analysis:** Analyze the "Scaling Lag"—how long does it take for a new pod to become "Ready" and start accepting traffic?
- **Metric:** Graph of Request_Rate vs. Active_Pod_Count.

### 129. Multi-Region Geo-Replication: "Global News Latency"

- **Dataset:** News_Articles replicated across 3 simulated "Regions" (US, EU, Asia).
- **The Task:** Implement a "Follow-the-Sun" write policy where the Primary node shifts based on the time of day.
- **Analysis:** Discuss the impact on **Global Consistency** during the Primary handover.
- **Deliverable:** A visualization showing the "Latency Heatmap" for users in different regions.

### 130. Serverless "Cold-Start" Data Persistence: "Event-Driven Analytics"

- **Dataset:** A stream of Clickstream_Events.
- **The Task:** Use a Serverless function that shuts down after 30 seconds of inactivity.
- **Analysis:** Measure the impact of "Cold Starts" on database connection pooling.
- **Metric:** Time to first byte ($TTFB$) for a warm function vs. a cold function.

Grading : Category 13

| Criteria | Excellent (90-100%) | Satisfactory (70-89%) | Developing (<70%) |
|---|---|---|---|
| **Containerization** | Uses multi-stage builds; optimized images; clear networking. | Basic Dockerfiles; bloated images; hard-coded IPs. | Manual installation only; no use of containers. |
| **Orchestration** | Flawless K8s StatefulSet or Compose logic with volume persistence. | Pods run but lose data on restart; manual scaling only. | Containers are isolated and cannot coordinate. |
| **Failover/Recovery** | Automated health checks and self-healing demonstrated. | System recovers but requires manual configuration/restart. | System stays down after a simulated container crash. |
| **Cloud-Native Metrics** | Analysis of "Cold Starts," Pod spin-up time, or Volume latency. | Basic "it works" description; no infrastructure metrics. | No consideration for cloud-specific overhead. |

# Category 14: Graph & Multi-Model Distributed DBs

### 131. Distributed Shortest Path: "Social Network Reach"

- **Dataset:** Social_Network_Edges (1 million connections).
- **The Task:** Partition a graph across 3 nodes. Implement Dijkstra's algorithm to find the shortest path between two users on different nodes.
- **Analysis:** Measure the overhead of "Cross-Shard Traversals."
- **Metric:** Number of network "Hops" vs. Local "Hops."

### 132. Community Detection in P2P: "Email Communication"

- **Dataset:** Enron_Email_Dataset (Nodes = Employees, Edges = Emails).
- **The Task:** Implement the **Louvain Method** for community detection in a distributed fashion.
- **Analysis:** Identify clusters of users without moving the entire graph to a single node.
- **Deliverable:** A visualization showing the identified "Departments" (communities).

### 133. Multi-Model Join: "Movies & Box Office"

- **Dataset:** Cast_Relationships (Graph) and Box_Office_Revenue (Relational).
- **The Task:** Perform a query: "Find all actors connected to Kevin Bacon who appeared in movies grossing $> \$100M$."
- **Analysis:** Compare a "Graph-first" vs. "SQL-first" join strategy.
- **Metric:** Total execution time for the cross-model query.

### 134. Cypher Query Optimizer: "Supply Chain Map"

- **Dataset:** A 5-level nested Supply_Chain_JSON (Factory -> Part -> Raw Material).
- **The Task:** Write a Cypher query (Graph SQL) to find all factories affected by a shortage of a specific raw material.
- **Analysis:** Show how the distributed graph engine "prunes" the search tree to avoid checking irrelevant nodes.
- **Deliverable:** An execution plan showing which shards were visited.

### 135. Graph Partitioning Strategies: "The METIS Test"

- **Dataset:** Citation_Graph (Papers and Citations).
- **The Task:** Compare **Random Partitioning** vs. **Edge-Cut Minimization** (METIS style).
- **Analysis:** How does the partitioning strategy affect query latency?
- **Metric:** The "Edge-Cut Ratio"—what percentage of edges span across two different nodes?

### 136. Distributed Graph Pattern Matching: "Fraud Ring Detection"

- **Dataset:** Financial_Transactions represented as a graph (Accounts as Nodes, Transfers as Edges).
- **The Task:** Search for a specific sub-graph pattern (e.g., a "Cycle" of 4 accounts moving the same money).
- **Analysis:** Explain how the query is distributed to avoid moving the entire graph to one node.
- **Deliverable:** A list of detected fraud "cycles" spanning multiple shards.

### 137. Temporal Graph Queries: "Historical Relationship Tracker"

- **Dataset:** Company_Board_Members (Who was on which board and when).
- **The Task:** Implement a "Time-Travel" graph query: "Find the network of influence for Person X as it existed in 2015."
- **Analysis:** Store edges with Start_Date and End_Date and filter them during traversal.
- **Metric:** Performance difference between a standard query and a temporal-filtered query.

### 138. Knowledge Graph Entity Resolution: "Data De-duplication"

- **Dataset:** Two separate graphs of Scientific_Papers from different sources.
- **The Task:** Identify and merge "Duplicate Nodes" (e.g., "J. Smith" and "John Smith") that exist on different distributed sites.
- **Analysis:** Use a "Similarity Score" for edges and nodes to decide on a merge.
- **Deliverable:** A "Clean" consolidated graph with identified cross-site links.

### 139. Distributed PageRank: "Web Page Importance"

- **Dataset:** A 100,000-node Web_Link_Graph.
- **The Task:** Implement the **PageRank algorithm** across 4 nodes. Nodes must exchange "Rank Scores" at the end of each iteration.
- **Analysis:** Measure the network traffic generated by "Rank Swapping" between nodes.
- **Metric:** Convergence rate (number of iterations) vs. Network Overhead.

### 140. Hybrid Document-Graph Store: "Medical Knowledge Base"

- **Dataset:** Patient_Symptoms (Documents) and Disease_Correlations (Graph).
- **The Task:** Build a system where a user queries a Document and the results are filtered by their "Graph Distance" to a specific medical field.
- **Analysis:** Evaluate the "Join Cost" between the Document engine and the Graph engine.
- **Deliverable:** A query result set that combines text-search relevance with graph-based importance.

Grading : Category 14

| Criteria | Excellent (90-100%) | Satisfactory (70-89%) | Developing (<70%) |
|---|---|---|---|
| **Graph Partitioning** | Uses advanced partitioning (METIS/Vertex-cut) to minimize hops. | Random partitioning leads to high cross-site traffic. | No partitioning; graph is stored on a single site. |
| **Traversal Logic** | Distributed implementation of BFS, DFS, or Shortest Path. | Traversal works but involves excessive redundant messaging. | Navigation fails when edges cross physical sites. |
| **Multi-Model Integration** | Seamless join logic between Graph and Relational/Document fragments. | Functional but requires significant manual data reformatting. | Models are siloed; cannot query across different types. |
| **Topology Analysis** | Deep insight into "Edge-Cut" ratios and cluster density. | Basic graph statistics provided (node/edge count only). | No analysis of graph structure influence on performance. |

# Category 15: AI-Driven & Future Trends

### 141. Learned Index Structures: "Neural B-Trees"

- **Dataset:** Sorted_Sensor_Data (Timestamps).
- **The Task:** Replace a traditional B-Tree index with a **Recursive Model Index (RMI)**—a neural network that predicts the location of a key.
- **Analysis:** Compare the memory footprint of the neural index vs. a standard index.
- **Metric:** $Memory\_Saved = \frac{Size_{BTree} - Size_{Neural}}{Size_{BTree}}$.

### 142. Self-Tuning Distributed DB: "Query Predictor"

- **Dataset:** Query_History_Logs.
- **The Task:** Train a simple ML model to predict the "Next Query." Based on the prediction, the system "Pre-fetches" data from Site B to Site A.
- **Analysis:** Measure the increase in **Cache Hit Rate**.
- **Deliverable:** A comparison of query response time with and without AI pre-fetching.

### 143. NLP-to-SQL for Distributed Fragments: "The Wiki-Query"

- **Dataset:** Wiki_Tables partitioned across 4 sites.
- **The Task:** Use an LLM API to convert a natural language question (e.g., "Which cities in Asia have the highest population?") into a localized SQL query.
- **Analysis:** The system must handle **Schema Mapping** automatically.
- **Metric:** Accuracy of the generated SQL vs. a manually written one.

### 144. Federated Learning for DBs: "Private Health Records"

- **Dataset:** 3 local "Hospital" databases with private patient data.
- **The Task:** Train a global ML model (e.g., disease predictor) without moving the patient data out of the hospitals.
- **Analysis:** Only the "Gradients" (weights) are shared between sites.
- **Deliverable:** Prove that the model reaches 90% accuracy without any raw data ever leaving the local databases.

### 145. Carbon-Aware Query Scheduling: "The Green Optimizer"

- **Dataset:** Cloud_Energy_Metrics (Real-time carbon intensity of AWS/Azure regions).
- **The Task:** Build a query optimizer that routes heavy batch jobs to the datacenter currently using the most renewable energy.
- **Analysis:** Analyze the trade-off between **Carbon Savings** and **Network Latency**.
- **Metric:** Total grams of $CO_2$ saved per 1,000 queries.

### 146. Reinforcement Learning for Partitioning: "The Smart Sharder"

- **Dataset:** A dynamic workload of E-commerce_Queries.
- **The Task:** Use a Reinforcement Learning (RL) agent to observe query patterns and suggest a new **Partitioning Key** every 1,000 queries.
- **Analysis:** Does the RL agent eventually outperform a static "Manual" partitioning strategy?
- **Metric:** Total "Cross-Node Joins" reduced over 10,000 queries.

### 147. Vector Database for Semantic Search: "AI Image Retrieval"

- **Dataset:** 5,000 Image_Embeddings (High-dimensional vectors).

- **The Task:** Implement a distributed **Approximate Nearest Neighbor (ANN)** search using HNSW or LSH.
- **Analysis:** Compare the accuracy of the distributed search vs. a single-node exhaustive search.
- **Deliverable:** A demonstration of "Searching by Image" across 3 distributed vector shards.

### 148. Self-Healing via Predictive Analytics: "Pre-emptive Failover"

- **Dataset:** Server_Hardware_Telemetry (CPU Temp, Disk Latency, Error Rates).
- **The Task:** Train a model to predict a "Node Failure" 5 minutes before it happens. Trigger an automatic 2PC-based data migration to a healthy node.
- **Analysis:** Evaluate the "False Alarm" rate vs. "Successful Migrations."
- **Metric:** Data availability % during a predicted vs. unpredicted failure.

### 149. Distributed Differential Privacy: "Income Inequality Study"

- **Dataset:** Employee_Salaries across 5 sites.
- **The Task:** Implement **Distributed Differential Privacy** where noise is added locally, but the *sum* of the noise cancels out to provide an accurate global average.
- **Analysis:** Use LaTeX to define the privacy budget: $\epsilon = 0.1$.
- **Deliverable:** A global report that is accurate to within 1% but keeps every individual salary private.

### 150. Quantum-Safe Distributed Commit: "Post-Quantum DB"

- **Dataset:** Government_Records.
- **The Task:** Implement a 2-Phase Commit (2PC) protocol where the "Prepare" and "Commit" messages are signed using **Post-Quantum Cryptography (PQC)** algorithms (e.g., CRYSTALS-Kyber).
- **Analysis:** Measure the computational overhead of PQC signatures compared to standard RSA/ECC.
- **Metric:** Transaction Latency (ms) with PQC vs. Traditional Encryption.

Grading : Category 15

| Criteria | Excellent (90-100%) | Satisfactory (70-89%) | Developing (<70%) |
|---|---|---|---|
| **AI/ML Integration** | Model significantly improves a DB function (e.g., Indexing/Tuning). | Model is implemented but provides negligible performance gain. | AI is a "gimmick" unrelated to the distributed DB logic. |
| **Training/Inference** | Correct use of Federated Learning or local inference protocols. | Data privacy is compromised during model training. | Model requires centralizing all data (defeats the purpose). |
| **Comparative Analysis** | Rigorous "AI vs. Heuristic" benchmarking (e.g., Neural vs. B-Tree). | Mentions improvements but lacks a controlled baseline. | No comparison against traditional DB methods. |
| **Future Vision** | Clearly addresses a modern challenge (Carbon, NLP, or Privacy). | Addresses a solved problem using "buzzwords" only. | Implementation lacks real-world utility. |

**Student Final Deliverable Checklist:**

1. **The Code:** A GitHub/GitLab repository with clear README instructions.
2. **The Analysis:** A report justifying design choices using **Özsu and Valduriez** theory.
3. **The Proof:** A screen-recording (3-5 mins) showing the system handling a specific dataset scenario.